

Production: Data Engineering Fundamentals

```
$ echo "Data Sciences Institute"
```

Agenda

2.1 Fundamentals of Data Engineering

- Data Sources
- Data Formats
- Data Models
- Data Storage and Processing
- Modes of Data Flow

Agenda (cont.)

2.2 An Initial Data Flow

- Jupyter notebooks and source code.
- Logging and using a standard logger.
- Environment variables.
- Getting the data.
- Schemas and index in Dask.
- Reading and writing parquet files.
- Dask vs pandas: a small example of big vs small data.

About

- These notes are based on Chapter 3 of *Designing Machine Learning Systems*, by Chip Huyen.

Our Reference Architecture

The Flock Reference Architecture

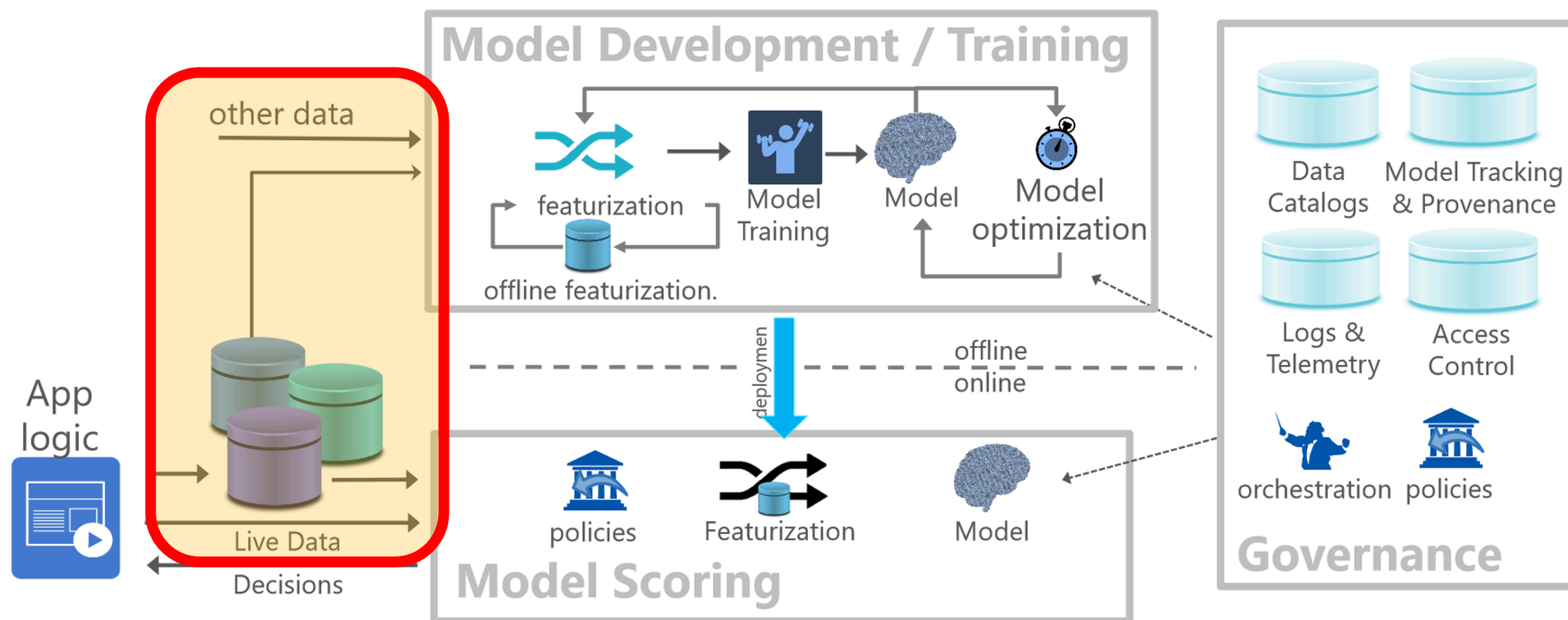
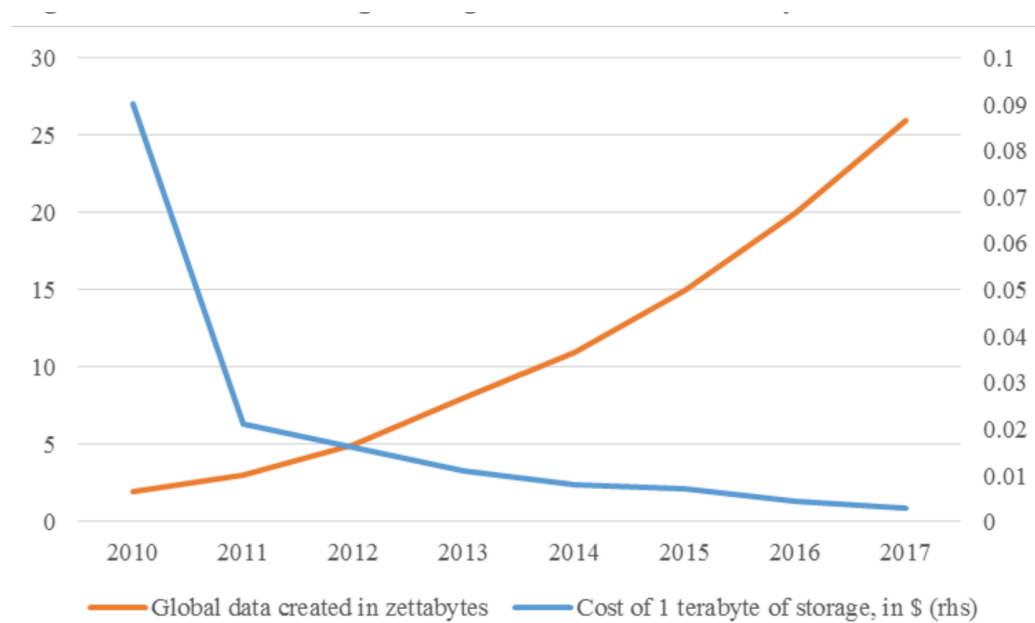


Figure 1: Flock reference architecture for a canonical data science lifecycle⁶

Data Sources

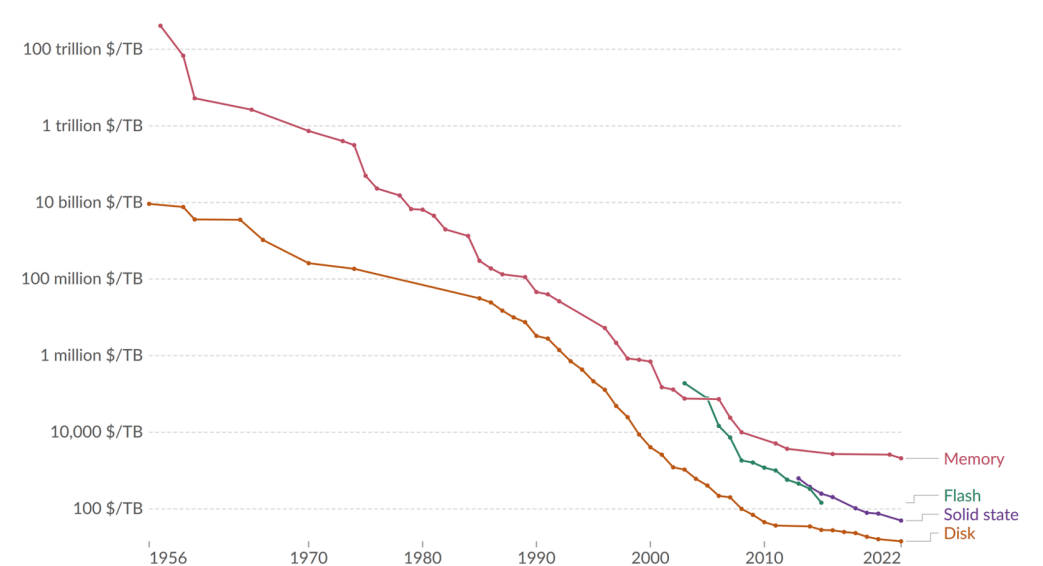
Why Now?



(FSB, 2017)

Historical cost of computer memory and storage

This data is expressed in US dollars per terabyte (TB). It is not adjusted for inflation.



Data source: John C. McCallum (2022)

OurWorldInData.org/technological-change | CC BY

Note: For each year, the time series shows the cheapest historical price recorded until that year.

(World In Data, 2024)

Data Sources (1/2)

- Different data sources have different characteristics.
- User input data:
 - Data that is explicitly input by users.
 - Text, images, videos, files, etc.
 - Prone to error: text too long, too short, incomplete, unexpected data types, etc.

Data Sources (2/2)

- System-generated data:
 - Logs, performance metrics, and other system outputs.
 - Generally well-formatted, and can grow rapidly.
- Databases generated by (internal) services and enterprise applications:
 - Many times, structured data.
 - Varying degrees of data quality.
- Third-party data:
 - Data collected from the public when the public is not a customer of the collecting organization.
 - Price databases, news aggregators, etc.

Data Formats

Data Formats (1/2)

- Data storage is a fundamental component in any ML system:
 - Store raw input data.
 - Store pre-computed features.
 - Store model performance metrics and other model-related information.
 - Store logs for monitoring and debugging.
- A sequence of operations reading from one or more storage types, combined with data transformation procedures, creates a *pipeline*.

Data Formats (2/2)

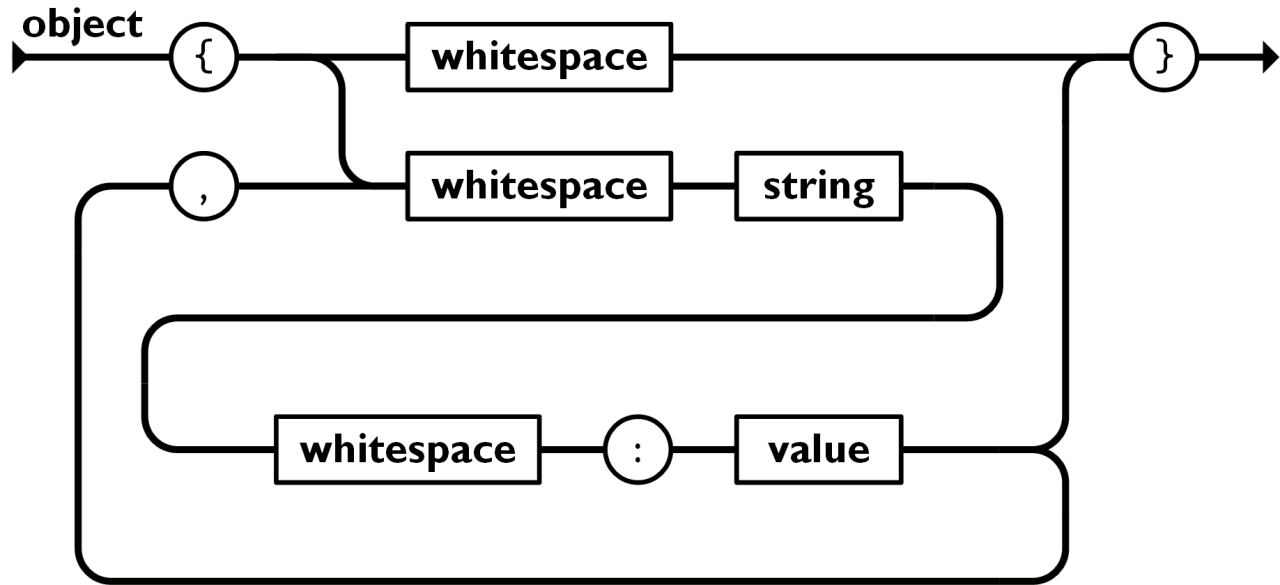
- Selecting the right data format for storing can be beneficial in terms of performance and costs.
- *Data serialization* is converting a data structure or object state into a format that can be stored, transmitted, and reconstructed later.
- Data formats can be:
 - Text or binary-based.
 - Human-readable.
 - Row-major or column-major.

Some Common Data Formats

Format	Binary/Text	Human-readable	Example use cases
JSON	Text	Yes	Everywhere
CSV	Text	Yes	Everywhere
Parquet	Binary	No	Hadoop, Amazon Redshift
Avro	Binary primary	No	Hadoop
Protobuf	Binary primary	No	Google, TensorFlow (TFRecord)
Pickle	Binary	No	Python, PyTorch serialization

JSON

- JavaScript Object Notation.
- Human-readable.
- Implements a key-value pair paradigm that can handle different levels of structured-ness.
- A popular format.
- Image: An illustration of an object in JSON (json.org, 2024)



JSON is Flexible (1/2)

Consider the record below.

```
{
  "firstName": "Boatie",
  "lastName": "McBoatFace",
  "isVibing": true,
  "age": 12,
  "address": {
    "streetAddress": "12 Ocean Drive",
    "city": "Port Royal",
    "postalCode": "10021-3100"
  }
}
```


JSON is Flexible (2/2)

We can also represent the data with less structure:

```
{  
  "text": "Boatie McBoatFace, aged 12, is vibing, at 12 Ocean Drive, Port Royal,  
  10021-3100"  
}
```

Row-Major vs Column-Major Formats

Row-Major Format

- Consecutive elements in a row are stored next to each other.
- Example: CSV (Comma-Separated Values in a text file).
- Accessing rows will tend to be faster than accessing columns.
- Faster for writing additional records.

Column-Major Format

- Consecutive elements in a column are stored next to each other.
- Example: parquet.
- Accessing columns will be faster than accessing rows.
- Faster for retrieving columns.

Row-Major vs Column-Major

Column-major:

- Data is stored and retrieved column by column
- Good for accessing features

Row-major:

- Data is stored and retrieved row by row
- Good for accessing samples

	Column 1	Column 2	Column 3
Example 1	...	...	...
Example 2	...	...	...
Example 3	...	...	...

(Huyen, 2022)

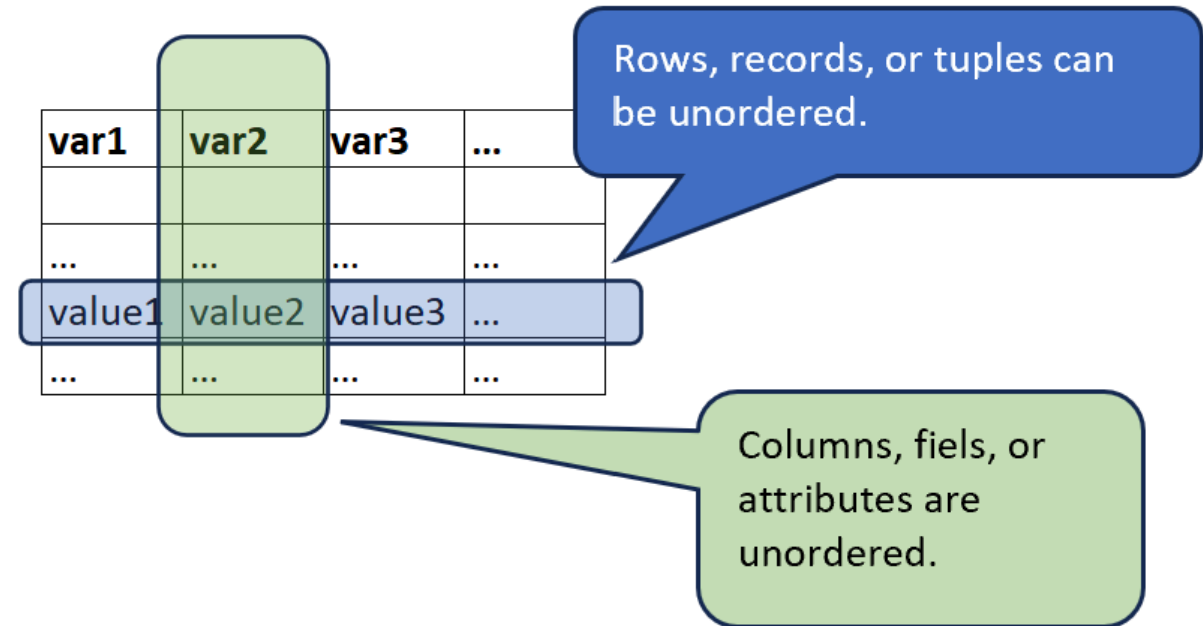
Text vs Binary Formats

- CSV and JSON files are stored as text files and are usually human-readable.
- Non-text file formats are called *binary*.
- Binary files are more compact:
 - To store the number 1000000 would require seven characters or 7 bytes (at one character per byte).
 - To store 1000000 as int32 would require 32 bits or 4 bytes.

Data Models

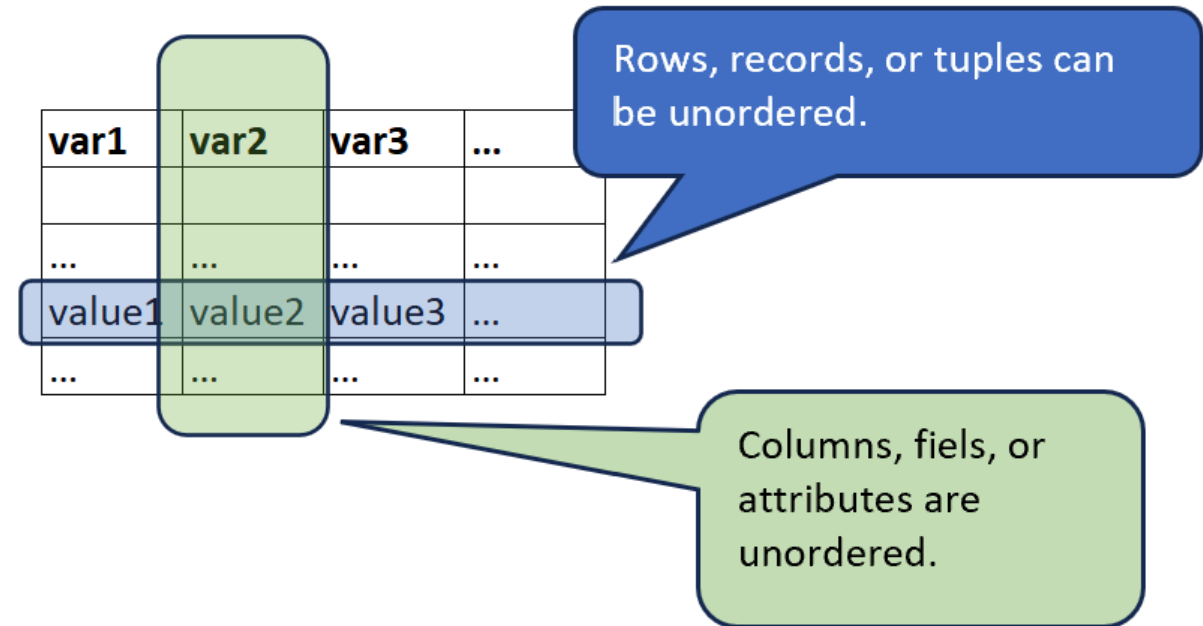
Relational Model

- Invented by Edgar F. Codd in 1970 in "A Relational Model of Data for Large Shared Data Banks"
- Data is organized into relations.
- Each relation is a set of tuples.



Relational Model

- A table is a visual representation of a relation: each relation is a set of tuples.
- Relations are unordered: we can shuffle rows or columns while retaining the relation.
- Data following the relational model are usually stored using CSV, parquet, and (some types of) databases.



Normalization



(Adapted from Huyen, 2022)

Normalization

- Normalization involves determining how much redundancy exists in a table and reducing it, as required.
- The goals of normalization are to:
 - Be able to characterize the level of redundancy in a relational schema.
 - Provide mechanisms for transforming schemas to remove redundancy.
- Generally, we want to minimize the redundancy of primary and foreign keys.
- One disadvantage of normalizing data is that it becomes spread out in different tables.

Query Language

- We use a query language to specify the data that we want from a database.
- SQL is the most popular query language.
- SQL is a declarative language.
- Optimizing queries is the hardest part.

Query Language (cont.)

- An *imperative language* requires the programmer to determine the steps that the program should follow (for example, Python).
- A *declarative language* requires the programmer to specify the output and the computer figures out the steps needed to get the queried outputs (for example, SQL).

No SQL

- The relational model applies to many use cases, but it can be restrictive: data needs to adhere to a schema.
- No SQL, started as a negation of SQL, but it is now generally understood as "Not Only SQL".
- No SQL models can be of two types:
 - Document model.
 - Graph model.

No SQL (cont.)

- The document model targets use cases in which data comes in self-contained units called documents. There is little relationship among the documents.
- The graph model targets use cases in which we try to identify common and important relationships.

Document Model (1/2)

- A document is most often a long continuous string, encoded as JSON, XML, or BSON (Binary JSON).
- All documents are encoded in the same format.
- Each document has a unique key that represents that document. We can use the key to retrieve the document.
- Schema-less: document does not enforce a schema.
- Schema-on-read: document databases shift the responsibility of assuming structures from the application that writes the data to the application that reads the data.
- Use cases: documents, images, video, audio, unstructured data.

Document Model (2/2)

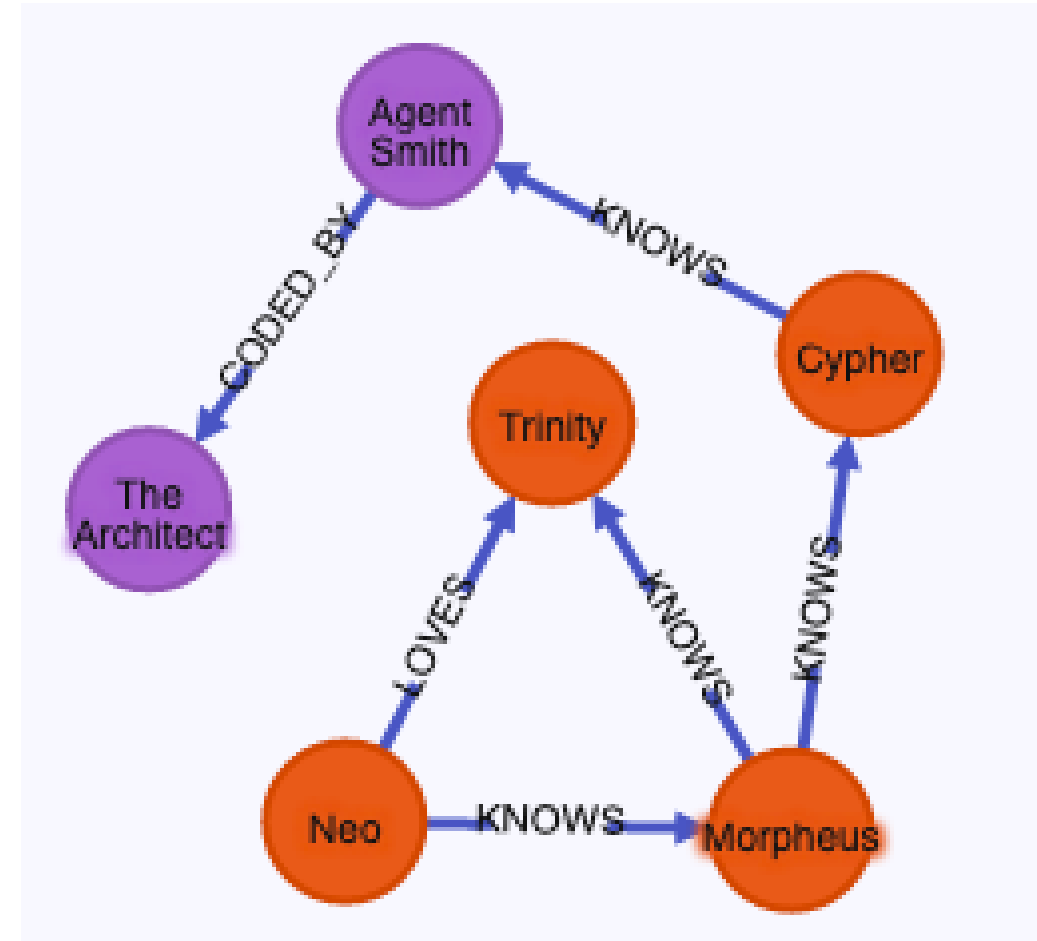
If a row in the relational model is somewhat equivalent to a document, then a table is equivalent to a collection of documents.

```
{
  "Title": "Harry Potter",
  "Author": "J.K. Rowling",
  "Publisher": "Banana Press",
  "Country": "UK",
  "Sold as": [
    {"Format": "Paperback", "Price": "$20"},
    {"Format": "E-book", "Price": "$10"}
  ]
}
```

Document1: harry_potter.json

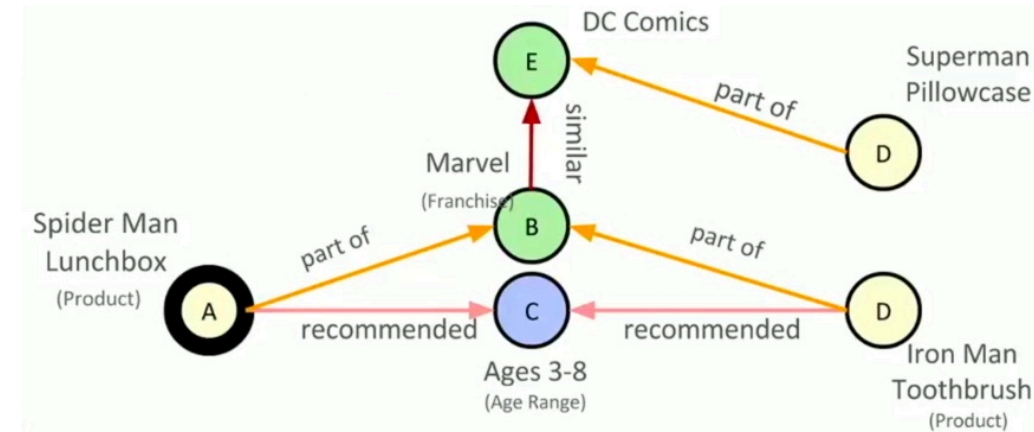
Graph Model

- A graph consists of nodes and edges.
- Edges represent relationships between nodes.
- In this model, the relationships between nodes are the priority.
- Faster to retrieve data based on relationships.
- Use cases: social interactions, payments, risk exposures, transmission, communications, connectivity, references.



Graph Model (cont.)

- Graph models enable network or graph metrics:
 - Node metrics like centrality measures: degree, eigen, and betweenness.
 - Graph-level features: cliques, clusters, modularity.
- Graph databases may bundle other features (visualization).



Structured vs Unstructured Data (1/3)

Data Model

- Structured data follows a predefined data model called a schema.
- Unstructured data can contain key-value pairs: even if we do not enforce a schema, the data may contain intrinsic patterns that help extract structures.

Schema

- Structured data follows a schema-on-write approach, where we commit to a predefined schema and use it to store the data.
- Unstructured data is schema-less in the sense that it is generally stored as documents. App developers can apply a schema to the data, an approach known as schema-on-read.

Structured vs Unstructured Data (2/3)

Storage

- Data warehouse: a repository for structured data.
- Data lake: a repository for unstructured data.

Structured vs Unstructured Data (3/3)

Structured data	Unstructured data
Schema clearly defined	Data does not need to follow a schema
Easy to search and analyze	Fast arrival
Can only handle data with a <u>specific schema</u>	Can handle data from <u>any source</u>
Schema changes will cause significant trouble.	<u>No need to worry about</u> schema changes (yet), as the worry is shifted to the downstream applications that use this data
Stored in <u>data warehouses</u>	Stored in <u>data lakes</u>

Data Storage and Processing

Data Storage and Processing

- Databases are storage engines that implement how data is stored and retrieved on machines.
- Typically, databases are optimized for transactional processing or analytical processing.
- A transaction is any action: buy/sell, a tweet, ordering a ride, uploading a new model, or watching YouTube.
- On-Line Transaction Processing (OLTP): transactions are inserted into the database as they are generated. Occasionally, they can be updated.

Data Storage and Processing (cont.)

- Transactional databases are designed to maintain low latency and high availability.
- Transactional databases usually offer ACID guarantees:
 - Atomicity: all steps in a transaction are completed successfully as a group. If one step fails, all fail.
 - Consistency: all transactions coming through must follow predefined rules.
 - Isolation: Two transactions happen at the same time as if they were isolated. Two users accessing the same data will not change it simultaneously.
 - Durability: once a transaction has been committed, it will remain committed even in the case of system failure.
- Some transactional databases do not offer ACID, but BASE: "Basically Available, Soft state, and Eventual consistency." (Kleppmann, 2017)

Transactional vs Analytical DB

- Because transactions are processed as a unit, transactional databases tend to be row-major. They will not generally be the most efficient for questions such as "What is the average price for all rides in September in San Francisco?"
- Analytical databases are efficient with queries that allow us to look at data from different viewpoints. They are usually called On-Line Analytical Processing (OLAP).

Transactional vs Analytical DB (cont.)

OLTP and OLAP are terms falling out of use, since the divide is somewhat outdated:

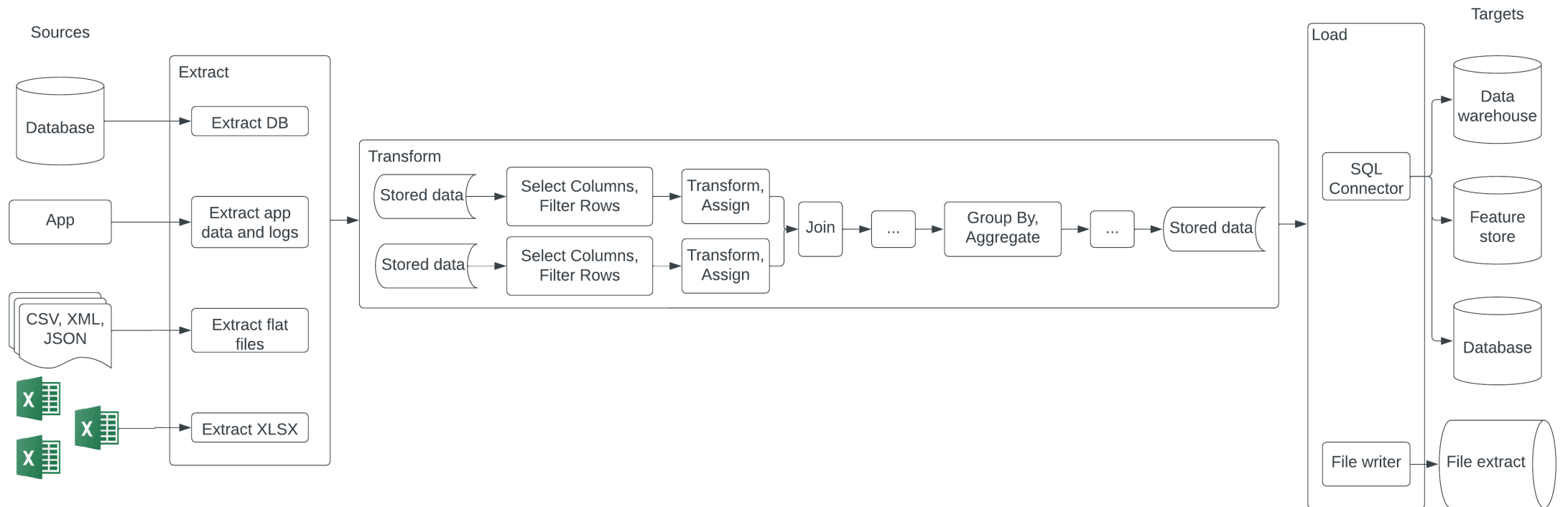
1. The separation was due to technological limitations: transactional databases that can handle analytical queries efficiently (e.g., CockroachDB)
2. Some solutions now decouple storage and compute (BigQuery, Snowflake, IBM, Teradata).
3. "Online" is now an overloaded term that can mean many things.

ETL: Extract, Transform, and Load

ETL is the process of extracting data from one or several sources, transforming it into the shape that an application or model requires, and loading it into a desired destination.

- Extract the data from all data sources, including validating and rejecting data that does not meet requirements. Notify sources of rejected data.
- Transform the data through different operations: join, filter, standardization, etc.
- Load is deciding how and how often to load the transformed data into the destination (a file, a database, or a data warehouse).
- Schema on read forces app developers to determine the schema in advance.
- Data acquisition grows rapidly, and storage is inexpensive.
- Some companies invested in a store-all-the-data strategy.

ETL Pipeline



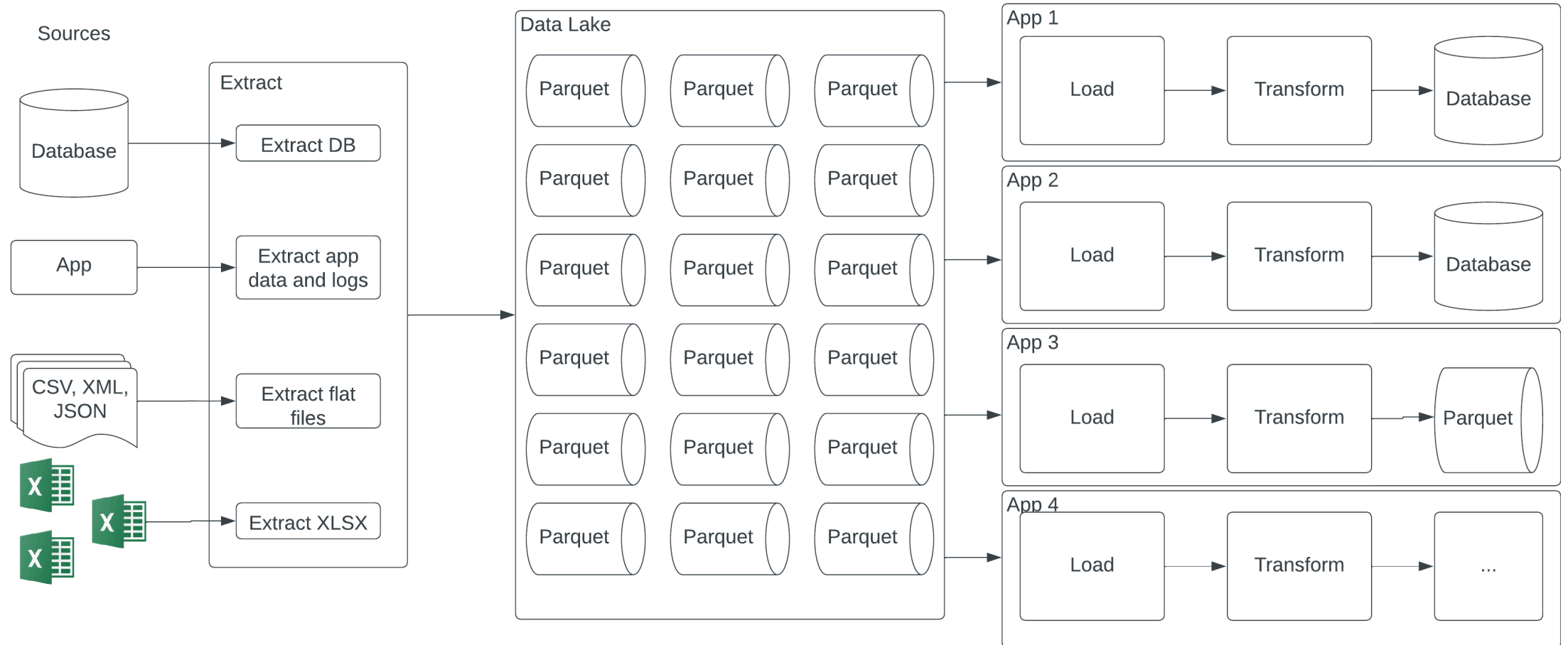
ELT: Extract, Load, and Transform

- Solves the problem of rapidly arriving data.
- Store first, figure out what to do with the data later.
- Use a compressed format.
- Take advantage of clusters of computers and the cloud.

ELT: Extract, Load, and Transform (cont.)

- Also difficult to manage.
- It is inefficient to search through a massive amount of raw data for your desired data.
- As infrastructure and frameworks become standardized, data is also becoming standardized.
- **Lakehouse solutions** (Databricks and Snowflake) **are hybrid solutions** that combine the flexibility of data lakes and the data management of data warehouses.

ELT Pipeline



Modes of Data Flow

Modes of Data Flow (1/2)

- Data 'flows' when data is passed from one process to another.
- In production, generally, we do not see data flows in the context of a single process. Instead, we find multiple processes.
- How do we pass data between processes that do not share memory?

Modes of Data Flow (2/2)

Three ways of passing data:

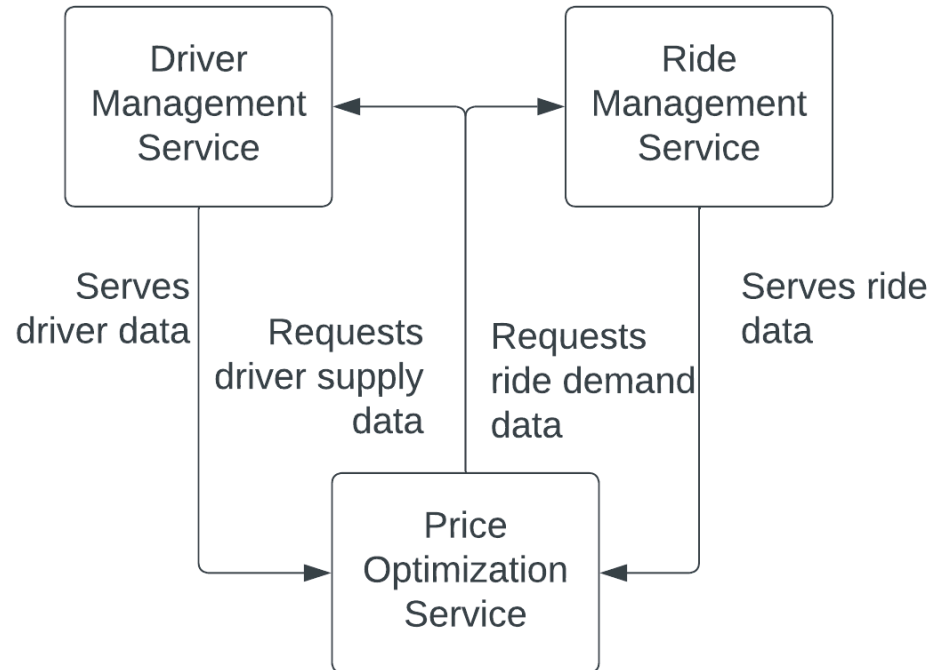
- Data passing through databases.
- Data passing through services using requests such as the requests provided by REST and RPC APIs (e.g., POST/GET requests).
- Data passing through a real-time transport like Apache Kafka and Amazon Kinesis.

Data Flows 1: Data Passing Through DBs



- Process 1 writes to the DB, Process 2 reads from the same DB.
- Both processes require access to the same database.
- Database access can be slow, which may not be suitable for apps with strict latency requirements, such as consumer-facing applications.

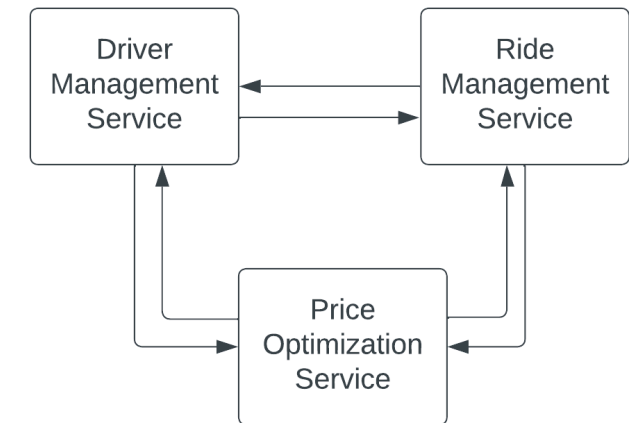
Data Flows 2: Data Passing Through Services



- Send data directly through a network: Process A requests data from Process B, which then returns it.
- Request-driven approach.
- Service-oriented architecture: Services can be from different companies or part of the same application (microservices).

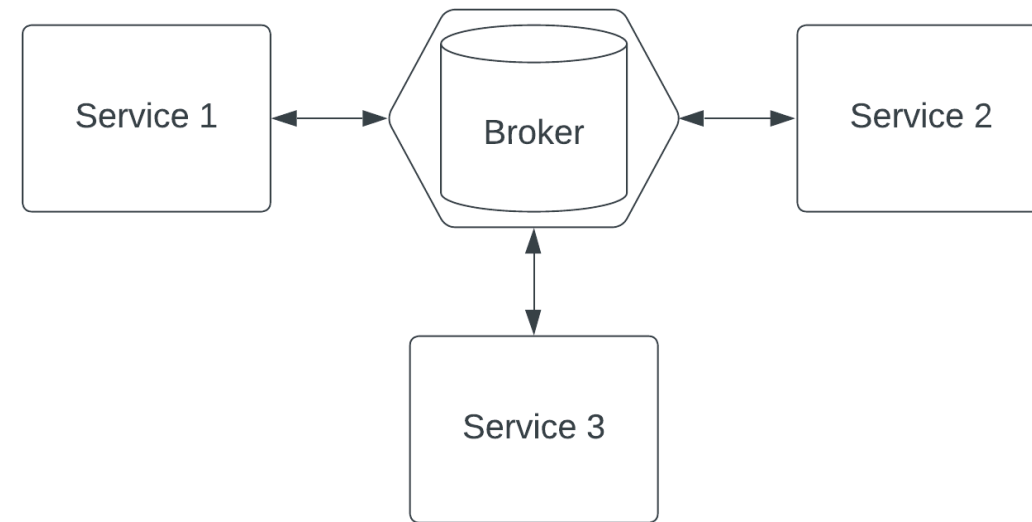
Data Flows 2: Data Passing Through Services (cont.)

- Popular communication frameworks include REST and RPC.
- REST is commonly used for public APIs and is implemented over HTTP.
- RPC aims to make service requests resemble internal function calls.
- The architecture can get complicated in complex scenarios with all services requesting and serving data.
- A central data broker may be a more efficient approach.



Data Flows 3: Data Passing Through Real-Time Transport

- Request-driven data passing is synchronous; the target service must be listening for the request.
- A broker can coordinate data passing among services, avoiding a complex web of interservice communication.
- Data produced by a service is passed to the broker for access by other services.
- Real-time transports like databases can introduce latency; in-memory transport is recommended for minimal delay.



Events and Types of Real-Time Transports

- A piece of data broadcast to a real-time transport is called an event.
- This architecture is called event-driven.
- The real-time transport is sometimes called an event bus.
- Request-driven architecture works well for systems that rely more on app logic than data.
- Event-driven architecture works better for data-intensive systems.

Events and Types of Real-Time Transports (cont.)

Two of the most common Real-Time Transports are:

- Pubsub (publish-subscribe).
- Message queue.

The Publish-Subscribe (pubsub) Model

In the pubsub model:

- Any service can publish to different topics in the real-time transport.
- Any service that subscribes to a topic can read all the events in that topic.
- There is a retention policy; for example, data will be retained for X days before being deleted or moved to permanent storage.
- Examples: Apache Kafka and Amazon Kinesis.

The Message Queue Model

In the message queue model:

- An event has intended consumers. An event with intended consumers is a message.
- The message queue is responsible for getting the message to the right consumers.
- Examples: Apache RocketMQ and RabbitMQ.

Batch and Stream Processing

Batch Processing

- Once data arrives in a data storage engine (database, data lake, or data warehouse, for example), it is historical data.
- Historical data is processed in batch jobs that are run periodically.
- Batch processing is a practice with mature solutions such as MapReduce and Spark.
- Batch processing is usually performed on slow-changing variables known as static features (for example, daily metrics).

Stream Processing

- Stream processing is performing computation on streaming data coming from real-time transports.
- Computation can also be started periodically, but the periods are generally shorter. Computation could also be started when needed.
- Stream processing is performed on rapidly changing variables known as dynamic features (for example, average metric in the past 5 minutes).
- Example products: Apache Flink, KSQL, and Spark Streaming.

References

References

- Agrawal, A. et al. "Cloudy with high chance of DBMS: A 10-year prediction for Enterprise-Grade ML." arXiv preprint arXiv:1909.00084 (2019).
- Huyen, Chip. "Designing machine learning systems." O'Reilly Media, Inc.(2022).
- Financial Stability Board (FSB). "Artificial intelligence and machine learning in financial services" (2017). [URL](#)
- Kleppmann, M. "Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems." O'Reilly Media, Inc. (2017).